

Chapter 08 : Deadlocks

Deadlock Avoidance

Banker's Algorithm

Banker's Algorithm

- The banker's algorithm is a resource allocation and deadlock avoidance algorithm.
 - It tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an "safe-state" check to test for possible activities, before deciding whether allocation should be allowed to continue.
-

Continue...

1. Banker's algorithm is named so because it is used in banking system to check whether loan can be sanctioned to a person or not.
 2. Suppose there are n number of account holders in a bank and the total sum of their money is S .
 3. If a person applies for a loan then the bank first subtracts the loan amount from the total money that bank has and if the remaining amount is greater than S then only the loan is sanctioned.
 4. It is done because if all the account holders comes to withdraw their money then the bank can easily do it.
 5. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers.
 6. The bank would try to be in safe state always.
-

Resource allocation for preventing deadlocks

1. **Multiple Instances:** Each resource type can have multiple instances available.
 2. **Claim Maximum Use:** Each process must declare, in advance, the maximum number of instances of each resource type it may need.
 3. **Request & Wait:** When a process requests resources, it may have to wait if those resources are currently unavailable.
 4. **Finite Hold Time:** Once a process has all the resources it needs, it must use them and then release them within a finite period.
-

Data Structures for the Banker's Algorithm

Let n = number of processes, m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If Max $[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If Allocation $[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If Need $[i,j] = k$, then P_i may need k more instances of R_j to complete its task.

$$\text{Need } [i,j] = \text{Max}[i,j] - \text{Allocation } [i,j]$$

Safety Algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = Available

*Finish [i] = **false** for $i = 0, 1, \dots, n-1$*

2. Find an i such that both:

(a) Finish [i] = false

(b) $Need_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$

$Finish[i] = true$

go to step 2

4. If $Finish [i] == \text{true}$ for all i , then the system is in a safe state

Resource-Request Algorithm for Process P_i

Request_i = request vector for process P_i .

If **Request_i[j] = k**, then process P_i wants **k** instances of resource type R_j

1. If **Request_i ≤ Need_i** go to step 2.

Otherwise, raise error condition, since process has exceeded its maximum claim

2. If **Request_i ≤ Available**, go to step 3.

Otherwise P_i must wait, since resources are not available

3. Pretend to allocate requested resources to P_i by modifying the state as follows:

Available = Available – Request_i;

Allocation_i = Allocation_i + Request_i;

Need_i = Need_i – Request_i;

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

Ans a) The content of the matrix **Need** is defined to be **Max – Allocation**

	<u>Need</u>
	A B C
P ₀	7 4 3
P ₁	1 2 2
P ₂	6 0 0
P ₃	0 1 1
P ₄	4 3 1

→ **Ans a)**

Answer the following questions using the banker's algorithm:

- a) What will be the content of the Need matrix?
- b) Is the system in a safe state? If Yes, then what is the safe sequence?
- c) What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Q2 : Is the system in a safe state? If Yes, then what is the safe sequence?

$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i = 0$ Step 2

Need₀ = 7, 4, 3 ✗

Finish [0] is false and Need₀ > Work ✗

So P₀ must wait But Need ≤ Work

For $i = 1$ Step 2

Need₁ = 1, 2, 2 ✓

Finish [1] is false and Need₁ < Work ✓

So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

A	B	C
5	3	2

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i = 2$ Step 2

Need₂ = 6, 0, 0 ✗

Finish [2] is false and Need₂ > Work ✗

So P₂ must wait

For $i = 3$ Step 2

Need₃ = 0, 1, 1 ✓

Finish [3] = false and Need₃ < Work ✓

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

A	B	C
7	4	3

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i = 4$ Step 2

Need₄ = 4, 3, 1 ✓

Finish [4] = false and Need₄ < Work ✓

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

A	B	C
7	4	5

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i = 0$ Step 2

Need₀ = 7, 4, 3 ✓

Finish [0] is false and Need < Work ✓

So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

A	B	C
7	5	5

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i = 2$ Step 2

Need₂ = 6, 0, 0 ✓

Finish [2] is false and Need₂ < Work ✓

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

A	B	C
10	5	7

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Finish [i] = true for $0 \leq i \leq n$ Step 4

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Question3. What will happen if process P_1 requests one additional instance of resource type A and two instances of resource type C?

$$\begin{array}{c} \text{A B C} \\ \text{Request}_1 = 1, 0, 2 \end{array}$$

To decide whether the request is granted we use Resource Request algorithm

$$\begin{array}{c} 1, 0, 2 \quad 1, 2, 2 \\ \text{Request}_1 < \text{Need}_1 \end{array} \quad \checkmark \quad \text{Step 1}$$

$$\begin{array}{c} 1, 0, 2 \quad 3, 3, 2 \\ \text{Request}_1 < \text{Available} \end{array} \quad \checkmark \quad \text{Step 2}$$

$\text{Available} = \text{Available} - \text{Request}_1$ $\text{Allocation}_1 = \text{Allocation}_1 + \text{Request}_1$ $\text{Need}_1 = \text{Need}_1 - \text{Request}_1$				Step 3
Process	Allocation	Need	Available	
	A B C	A B C	A B C	
P_0	0 1 0	7 4 3	2 3 0	
P_1	3 0 2	0 2 0		
P_2	3 0 2	6 0 0		
P_3	2 1 1	0 1 1		
P_4	0 0 2	4 3 1		

We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.

Step 1 of Safety Algo

$m=3, n=5$
 Work = Available

Work =

2	3	0
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

Step 2:

For $i=0$
 $Need_0 = 7, 4, 3$
 Finish [0] is false and $Need_0 > Work$
 So P_0 must wait

But $Need \leq Work$

Step 2:

For $i=1$
 $Need_1 = 0, 2, 0$
 Finish [1] is false and $Need_1 < Work$
 So P_1 must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

Step 2:

For $i=2$
 $Need_2 = 6, 0, 0$
 Finish [2] is false and $Need_2 > Work$
 So P_2 must wait

Step 2:

For $i=3$
 $Need_3 = 0, 1, 1$
 Finish [3] is false and $Need_3 < Work$
 So P_3 must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

Step 2:

For $i=4$
 $Need_4 = 4, 3, 1$
 Finish [4] is false and $Need_4 < Work$
 So P_4 must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

Step 2:

For $i=0$
 $Need_0 = 7, 4, 3$
 Finish [0] is false and $Need_0 < Work$
 So P_0 must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

Step 2:

For $i=2$
 $Need_2 = 6, 0, 0$
 Finish [2] is false and $Need_2 < Work$
 So P_2 must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$
 Hence the system is in Safe state

The safe sequence is P_1, P_3, P_4, P_0, P_2

Hence the new system state is safe, so we can immediately grant the request for process P_1 .

Practice

- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

Deadlock vs. Starvation

Features	Deadlock	Starvation
Definition	Deadlock happens when every process holds a resource and waits for another process to hold another resource.	Starvation happens when a low priority program requests a system resource but cannot run because a higher priority program has been employing that resource for a long time.
Basic	A deadlock occurs when no process can proceed and becomes blocked.	Starvation occurs when low priority procedures are blocked while high priority operations proceed.
Other names	Deadlock is also known as circular wait.	Starvation is known as a Lived lock.
Resources	Other processes block requested resources while a process is deadlocked.	High-priority processes continue to use the requested resources.
Arising Condition	Mutual exclusion's occurrence, Hold and wait, No preemption, and Circular wait all happen simultaneously.	Uncontrolled resource management, enforcement of priorities.
Prevention	It can be prevented by avoiding the situations that lead to deadlock.	Aging may prevent it.

****Aging** is a method that gradually increases the priority of processes that wait in the queue for a long time. This ensures that no process is starved of resources and eventually gets its turn.

Strategies for handling Deadlock

Deadlock Ignorance:

- Deadlock Ignorance is the most widely used approach among all the mechanism. This is being used by many operating systems mainly for end user uses. In this approach, the Operating system assumes that deadlock never occurs. It simply ignores deadlock. This approach is best suitable for a single end user system where User uses the system only for browsing and all other normal stuff.
 - There is always a tradeoff between Correctness and performance. The operating systems like Windows and Linux mainly focus upon performance. However, the performance of the system decreases if it uses deadlock handling mechanism all the time if deadlock happens 1 out of 100 times then it is completely unnecessary to use the deadlock handling mechanism all the time.
 - In these types of systems, the user has to simply restart the computer in the case of deadlock. Windows and Linux are mainly using this approach.
-

Strategies for handling Deadlock

Deadlock prevention

- Deadlock happens only when Mutual Exclusion, hold and wait, No preemption and circular wait holds simultaneously. If it is possible to violate one of the four conditions at any time then the deadlock can never occur in the system.
- The idea behind the approach is very simple that we have to fail one of the four conditions but there can be a big argument on its physical implementation in the system.

Deadlock avoidance

- In deadlock avoidance, the operating system checks whether the system is in safe state or in unsafe state at every step which the operating system performs. The process continues until the system is in safe state. Once the system moves to unsafe state, the OS has to backtrack one step.
 - In simple words, The OS reviews each allocation so that the allocation doesn't cause the deadlock in the system.
-

Strategies for handling Deadlock

Deadlock detection and recovery

- This approach let the processes fall in deadlock and then periodically check whether deadlock occur in the system or not. If it occurs then it applies some of the recovery methods to the system to get rid of deadlock.

THANK YOU